

# Graphs & Complexity Analysis

Complete Handwritten Lecture Notes & Solved Problems

---

## Part II: Graphs & Complexity Analysis

Graph storage, DFS, BFS traversals, topological sorting, shortest path algorithms (Dijkstra, Floyd), and minimum spanning trees.

PAGE 1

---

linear non-linear

linear list tree graph logical structure

ordered list linked list index hash memory structure

- search; insert; delete; merge; split; traverse
- string search
- sort: exchange: bubble, quick
- insertion: ..., shell
- selection:
- merge:
- heap: ..., tree selection, tournament

- Introduction

## 1.1 data abstraction & binary relation

### a. data abstraction

element + relation  $\rightarrow$  directed, weakly connected graph

### b. Cartesian product

$$A \times B = \{(a, b) \mid a \in A \ \& \ b \in B\}$$

$$A_1 \times \dots \times A_n = \{(a_1, \dots, a_n) \mid a_i \in A_i\}$$

### c. binary relation

$R$  is a subset of  $A \times B$ , called an endorelation over  $A$

when  $A = B$ .

reflexivity:  $(a, a) \in R$ , symmetry:  $(a, b) \in R \Rightarrow (b, a) \in R$

## 1.2 Basics of data structure

### a. Logical structure of data

$B = (D, R)$  D: data, R: endorelation

### b. Memory structure of data

see page 1

### c. Abstract data type

data structure + operations

## 1.3 Algorithms

### a. Brute force:

b. Divide and conquer: merge sort, quick sort, shell sort.

c. Decrease and conquer: binary search, b-tree search.

d. Backtracking: maze

e. Greedy: KMP, Prim, Kruskal, Dijkstra, Huffman

f. Dynamic programming: Floyd-Warshall

- Linear list

$B = (D, R)$   $D = \{a_i \mid i = 1, \dots, n\}$ ,  $R = \{(a_i, a_{i+1}) \mid i = 1, \dots, n-1\}$

## 2.1 Ordered list

## 2.2 Stack

double stack:

e.g. recursion + divide & conquer

$$T(n) = aT(n/b) + f(n)$$

if (P solvable) solve P directly

else for (i = 1: k)  $y_i = \text{Divide}(P_i)$  // solve subproblems

## 2.3 Queue

### a. sequential

$$\uparrow a_1 a_2 \dots a_n \rightarrow \nabla$$

front rear

### b. circular

delete: front = rear = 0

enqueue: (rear + 1) % ms

dequeue: (front + 1) % ms

empty: front == rear

full: (rear + 1) % ms == front

### c. priority: heap

## 2.4 Array and matrix

a. row-based array:  $AD(a_{ij}) = AD(a_{11}) + [(i-1)n + j-1] \text{ unit};$

column-based array:  $AD(a_{ij}) = AD(a_{11}) + [(j-1)n + i-1] \text{ unit}$

### b. compressed matrix

- lower-triangular:

$$\text{row: } k = \frac{1}{2}i(i-1) + j$$

$$\text{col: } k = \frac{[2n - (i-1)]j}{2} + i - (j-1)$$

- symmetric

- lower-triangular:

$$\text{row: } k = \begin{cases} \frac{1}{2}i(i-1) + j & i \geq j \\ \frac{1}{2}j(j-1) + i & i < j \end{cases}$$

- diagonal:

$$\text{row: } k = \begin{cases} (\lfloor \frac{m}{2} \rfloor + 1) \cdot i + j - \lfloor \frac{m}{2} \rfloor + 1 & \text{else} \\ 0 & \text{if not}(i \leq j \&\& j \leq i + \lfloor \frac{m}{2} \rfloor) \&\& (i \leq j) \end{cases}$$

c. sparse matrix: B(row, col, val)

- POS[k]: first non-zero element in row k -  $\text{POS}[i] = \text{POS}[i-1] + \text{NUM}[i-1]$

- NUM[k]: number of non-zero elements in row k:  $\text{NUM}[B[i, 1]] = \text{NUM}[B[i-1]] + 1$

3. linked list

### 3.1 regular linked list

\*insertion: empty list; insert to the 1st node; can't find where to insert

deletion: empty list; delete the 1st node; can't find where to delete

merge: choose a base list,  $O(n+m)$

split: odd pos/even pos =  $O(n)$

### 3.2 linked stack, linked queue

#### a. linked stack

top  $\rightarrow [] \rightarrow [] \rightarrow \dots \rightarrow []$

#### b. linked queue

$[] \rightarrow [] \rightarrow \dots \rightarrow []$

front rear

### 3.3 circular linked list

$H \rightarrow [] \rightarrow [a1] \rightarrow \dots \rightarrow [an] \rightarrow H$

empty:  $H \rightarrow []$  : no need to single out the empty case (vs 3.1)

check rear:  $p \rightarrow \text{next} == \text{head}$

merge:  $p1 \rightarrow \text{next} = \text{head2}$ ,  $p2 \rightarrow \text{next} = \text{head1}$

split: create new head, consider odd pos/even pos, choose base

### 3.4 multi-linked list

### 3.5 lists

linear; recursive definition; depth = # of layers of recursion

length = # of "next"

e.g. head:

| flag=0 | sublist | next |

|-----|-----|-----|

| flag=1 | data | next |



- Tree; Binary Tree

## 4.1 Basics of Tree

$n$ : number of nodes, degree of node  $i$ :  $d_i \Rightarrow n = \sum d_i + 1$

$k$ : branches of a node (maximum) = least depth =  $\lceil \log_2[n(k-1)+1] \rceil$

## 4.2 Binary Tree

### a. Basic Properties

- No leaves,  $n_2$  nodes with degree 2  $\Rightarrow n_0 = n_2 + 1$

proof: let  $n$  be # of nodes,  $n_1$  be # of nodes with degree 1

$$n = n_0 + n_1 + n_2$$

every node has a parent except the root:

$$n - 1 = 2n_2 + n_1$$

$$n_0 = n_2 + 1$$

- Full (all leaves present) vs balanced vs complete (leaves present left-right)

$n$  nodes  $\Rightarrow$  depth =  $\lceil \log_2 n \rceil + 1$

depth  $h \Rightarrow n \leq 2^h - 1$

- Left child of  $i = 2i + 1$ , right child of  $i = 2i + 2$

parent:  $\lfloor (i-1)/2 \rfloor$

left child of  $i$ :  $2i$ , right child of  $i$ :  $2i + 1$

parent:  $\lfloor i/2 \rfloor$

### 4.3 Memory Structure of Binary Tree

linked list; complete binary trees can be stored using a heap

### 4.4 DFS, time: $O(n)$ , space: $O(\log_2 n) \sim O(n)$

can also be implemented in a non-recursive way

DFS: visit  $\rightarrow l \rightarrow r$

DFS: visit  $\rightarrow r \rightarrow l$

DFS:  $l \rightarrow r \rightarrow \text{visit}$

DFS: visit  $\rightarrow l \rightarrow r$

DFS: visit  $\rightarrow r \rightarrow l$

DFS:  $l \rightarrow r \rightarrow \text{visit}$

DFS: visit  $\rightarrow l \rightarrow r$

DFS: visit  $\rightarrow r \rightarrow l$

DFS:  $l \rightarrow r \rightarrow \text{visit}$

4.5 BFS binary tree time:  $O(n)$  space:  $k^{(h-1)}$ ,  $k$ : degree,  $h$ : depth

Level order tree traversal: for getting depth, deletion, getting # of nodes while  
 (!EmptyQueue()) { enqueue left child; enqueue right child; dequeue; }

4.6 threaded binary tree for  $O(1)$  space traversal:

$l\_tag$ :  $l\_kid$ : data:  $r\_kid$ :  $r\_tag$

threaded preorder: thread cur\_node & preorder predecessor  $\rightarrow l \rightarrow r$

threaded in order:  $l \rightarrow$  thread cur\_node & in order predecessor  $\rightarrow r$

threaded post order:  $l \rightarrow r \rightarrow$  thread cur\_node & postorder predecessor predecessor  $\rightarrow$   
 $l\_kid \rightarrow r\_kid$ .

4.7 counting in binary trees

number of in order traversals corresponding to a preorder:  $\frac{1}{n+1} C_2^n$

to specify a binary tree: using preorder to determine root, in order kids.

4.8 applications of binary trees

a. ordered tree  $\rightarrow$  binary linked list  $\rightarrow$  binary tree

b. optimal binary tree (Huffman tree)

depth  $h$ :  $2^{h-1} \leq n \leq 2^h - 1$

$$\{ n_0 = n_2 + 1 \} \{ n_0 = \frac{n+1}{2} \}$$

$$\{ n_0 + n_2 = n \} \{ n_2 = \frac{n-1}{2} \}$$

minimize:  $\sum_{i=1}^n \text{weight}(i) * \text{length}(i)$

algorithm: select the two trees from the forest with minimum weights to form a binary tree; greedy

root weight = l\_weight + r\_weight

### C. Binary search tree BST

depth  $h$ :  $h < n < 2^h - 1$

- $N_{\text{left}} < N_{\text{right}}$  : in order insertion, time, space:
- deletion { leaf: delete directly ( $\log n$ )  $\sim O(n)$

degree 1 node: delete following child

degree 2 nodes: replace with in order predecessor  $\rightarrow$  delete in order predecessor { leaf ...  
degree 1 ...

d. Heap (binary sort) findmin/findmax in  $O(1)$

depth  $h$ :  $2^{(h-1)} \leq n \leq 2^h - 1$

{ max-heap: parent  $>$  kid } list, complete binary tree

{ min-heap: parent  $<$  kid } index by layer

- insertion: insert to rear (lower right), sift up the element:  $O(\log n)$
- deletion: { directly delete if at rear

swap top & rear, sift down the top:  $O(\log n)$  if at top

- creation: { insert each element (sift up):  $O(n \log n)$   $\rightarrow$  +

(sorting) sift down each element from lower right to upper left:  $O(n)$

e. Solution space tree

backtracking: pre-order traversal of the solution space tree

### 4.9. Equivalence class and union-find (disjoint-set)

memory structure: multi-branch linked list-based tree

{ make set: rank=0, parent=x

{ find: x.parent = find (x.parent) : recursive

union: make the higher-ranked parent as parent.

- graph

## 5.1 representation

$$G = (V, E)$$

$$G' \subseteq G$$

if

$$G'$$

is a subgraph of

$$G$$

undirected: degree = edges connected

directed: degree = inbound edges + outbound edges

$$\sum_{v \in V} \deg(v) = 2|E|$$

$$n - 1 \leq |E| \leq C_n^2$$

for undirected graphs

$$n \leq |E| \leq 2C_n^2$$

for directed graphs

## 5.2 memory

a. adjacency matrix for dense graphs

inbound degree =

$$\sum_{i=0}^{n-1} \text{col}_i = \sum_{i=0}^{n-1} A[i][j]$$

outbound degree =

$$\sum_{j=0}^{n-1} \text{row}_j = \sum_{j=0}^{n-1} A[i][j]$$

b. linked list for sparse graphs

triplet representation: (row, column, value)

linked representation: (row, column, value, next node)

### 5.3 traversal

a. DFS (DFT): adjacency matrix:

$$O(n^2)$$

, linked list:

$$O(n + e)$$

, space:

$$O(n)$$

≈ pre-order traversal of a tree

b. BFS (BFT): adjacency matrix:

$$O(n^2)$$

, linked list:

$$O(n + e)$$

, space:

$$O(n)$$



$\approx$  level order traversal of a tree (single source single sink shortest path in an unweighted graph)

#### 5.4 MST minimum spanning tree (min

$$\sum_{i \in E} e_i$$

,

$$E \in \text{MST}$$

)

$$V(G') = V(G)$$

, n vertices, n-1 edges.

a. Prim:

$$O(n^2)$$

: greedy, use heap

b. Kruskal:

$$O(e \log e)$$

: greedy, use union-find structure

## 5.5 Single source shortest path (all sinks)

- a. w/o negative weights: Dijkstra, based on priority queue  $O((V + E) \log V)$  faster than iterative DFS's
- b. w/ negative weights: Bellman-Ford, slower than Dijkstra  $O(V * E)$
- c. SPFA: Shortest path faster algorithm w/ negative weights  $O(k * E)$ ,  $k \ll V$

## 5.6 acyclic shortest path, DAGs

$O(V + E)$

## 5.7 topological sorting, DAGs

$O(V + E)$ , using DFS

## 5.8 all-source shortest path (all sinks), w/ negative weights

$O(V^3)$ , Floyd-Warshall, dynamic programming

- Search, string search

6.1 binary search tree. see 4.8, C, worst case  $O(n)$  for search, insert, delete and space.

## 6.2 self-balancing binary search tree

- a. AVL tree { worst case  $O(\log n)$  for }
- b. red-black tree search, insert, delete.

## 6.3 high fan-out self-balancing tree

- b. Knuth-Morris-Pratt: preprocessing:  $O(m)$ , matching:  $O(n)$ , space:  $O(m)$

## 6.4. string searching algorithm

string:  $n$  > pattern:  $m$

a. straightforward:  $O(m * n)$

- Sort

## 7.1 Exchange sort: swap

- bubble sort -> bidirectional bubble sort  $O(n^2)$
- quick sort: Choose the median of  $R[m]$ ,  $R[m+n]$ ,  $R[n]$ , divide and conquer, recursion  $O(\log n) \rightarrow O(n^2)$

## 7.2 Insertion sort: insert to sorted list

- naive  $O(n^2)$
- binary search the sorted list  $O(n \log n) \rightarrow O(n^2)$  or swaps
- shell  $O(n^{1.5})$

## 7.3 Selection sort: select from unsorted list

- naive  $O(n^2)$
- heap sort: time  $O(n) - O(n \log n)$ , space:  $O(1)$

see 4.8 d, divide and conquer

## 7.4 Merge sort: divide and conquer, $O(n \log n)$

{ top-down  $O(n)$  space: external sort

bottom-up easy to parallelize, can be used on disk

## 7.5 Distribution sort (non-comparison sort) integers

- bucket sort time:  $O(n + k) - O(n^2)$ , space:  $O(n \cdot k)$
- counting sort time:  $O(n + k)$ , space:  $O(n + k)$

c. radix sort time:  $O(wN)$ , space:  $O(w + N)$